

Middleware authentication dan middleware authorization

1. What is **authentication**?

- Authentication adalah sebuah proses memverifikasi user yang dijalankan pertama kali ketika masuk ke dalam website.
- Authentication berhubungan dengan authorization karena bersambungan prosesnya, authentication menjadi proses pertama yang akan dijalankan, sementara authorization menjadi proses selanjutnya.
- Real case use: Authentication banyak digunakan pada sistem **sign in, sign up**.

2. What is **authorization**?

- Authorization adalah sebuah proses memverifikasi hak akses yang dimiliki oleh seorang user.
- Authorization dilakukan setelah authentication karena user harus terlebih dahulu di verifikasi identitasnya melalui authN, baru di verifikasi hak akses nya melalui authZ.
- Real case use: Authorization banyak digunakan pada pengecekan role, seperti **sistem pemilik data ataupun admin/user**

Alur Kerja dari Autentikasi Modern

1. **USER LOGIN**, client mengirimkan email dan password melalui POST /api/login.
2. **VERIF PASSWORD**, server cocokin passwordnya dengan password yang sudah dienkripsi pada database.
3. **JSON WEB TOKEN**, setelah input cocok dengan database, server dapat mengirimkan sebuah token yang berisi identitas user dan jam dibuatnya token tersebut, contoh JWT di bawah

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJlcm9sZSI6ImN1c3RvbWVyIiwiaWF0IjoxNzEzNDU2Nzg5fQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

4. **Penyimpanan token**, token kemudian disimpan pada fitur local storage atau cookie, agar user dapat terus menggunakan website.
5. **Request selanjutnya**, setiap kali client mau meminta sebuah data, client/browser perlu mengirimkan JWT di Header Request.
6. **Authz**, setelah token yang diberikan cocok. Maka server memperbolehkan user dalam menggunakan website sesuai dengan hak akses yang ia miliki.

Real use

1. `jwt.sign`

- Fungsi `jwt.sign()` digunakan pada endpoint **POST** `/api/login` untuk membuat string token terenkripsi setelah user berhasil memverifikasi **username** dan **password**.

```
JavaScript

const payload = {
  userId: user.id,
  username: user.username,
  role: user.role
};

const tokenBaru = jwt.sign(payload, secretkey, { expiresIn: '1h' });
```

payload (Object):

- **Fungsi:** Data identitas user yang ingin dititipkan/disimpan di dalam token.
- **Penjelasan:** Di sini kita menyimpan **userId**, **username**, dan **role**. Data ini nantinya bisa dibaca kembali oleh server tanpa perlu melakukan query ulang ke database.

secretkey (String):

- **Fungsi:** Kunci rahasia milik server untuk menandatangani (*sign*) token.
- **Penjelasan:** Hanya server yang tahu kunci ini. Kunci inilah yang membuat token tidak bisa dipalsukan oleh hacker.

{ expiresIn: '1h' } (Options Object):

- **Fungsi:** Menentukan masa berlaku (*expiration time*) token.
- **Penjelasan:** **'1h'** artinya token hanya berlaku selama 1 jam sejak diterbitkan. Jika sudah lewat 1 jam, token otomatis kadaluarsa dan user wajib login ulang.

2. Middleware Autentikasi (**autentikasi_user**)

- Middleware ini bertugas memverifikasi apakah client mengirimkan token yang sah pada setiap request yang terproteksi.

JavaScript



```
const autentikasi_user = (req, res, next) => {
  const authHeader = req.headers['authorization'];
  if (!authHeader) {
    return res.status(401).json({ error: "Token tidak ditemukan" });
  }

  const token = authHeader.split(' ')[1];
  if (!token) {
    return res.status(401).json({ error: "Format token salah!" });
  }

  jwt.verify(token, secretkey, (err, decodedData) => {
    if (err) {
      return res.status(401).json({ error: "Token tidak valid atau expired" });
    }
    req.user = decodedData;
    next();
  });
};
```

`req.headers['authorization']`:

- Membaca header `Authorization` dari request client. Nilai aslinya berbentuk string: `"Bearer <token_panjang>"`.

`authHeader.split(' ')[1]`:

- Memotong string berdasarkan spasi (' ') menjadi Array `["Bearer", "<token_panjang>"]`.
- Indeks `[1]` diambil untuk membuang kata `"Bearer"` dan mengambil **pure string** token-nya saja.

`jwt.verify(token, secretkey, callback)`:

- **Fungsi**: Menguji keabsahan token menggunakan `secretkey` milik server.
- **Parameter Callback (err, decodedData)**:
 - **err (Error Object)**: Bernilai *truthy* jika token salah, rusak, atau sudah kadaluarsa (`expiresIn` habis). Jika ada error, server langsung menghentikan request dengan response `401 Unauthorized`.
 - **decodedData (Object)**: Hasil ekstrak/dekripsi dari token jika valid. Isinya persis sama dengan **payload** yang dibuat saat `jwt.sign()`.

`req.user = decodedData`:

- **Fungsi**: Menempelkan objek hasil dekripsi ke properti `req.user`.

- **Penjelasan:** Karena `req` adalah objek yang terus dialirkan antar-middleware, menempelkan data di `req.user` membuat middleware/route selanjutnya bisa mengetahui identitas user yang sedang mengakses.

`next()`:

- **Fungsi:** Memerintahkan Express untuk melanjutkan alur eksekusi ke middleware atau handler route berikutnya.

3. Middleware Otorisasi (**Authorization**)

- Middleware ini bertugas mengecek apakah user yang sudah terautentikasi memiliki hak akses (*role*) yang sesuai untuk menjalankan operasi tertentu.

```
JavaScript
const Authorization = (req, res, next) => {
  if (req.user && req.user.role === "admin") {
    next();
  } else {
    return res.status(403).json({
      error: "Akses Ditolak! Hanya Admin yang bisa mengakses."
    });
  }
};
```

`req.user.role === "admin":`

- **Fungsi:** Membaca properti `role` dari `req.user` yang sebelumnya disuntikkan oleh `autentikasi_user`.
- **Penjelasan:** Sistem mengecek secara spesifik apakah user memiliki tingkat akses `"admin"`.

Percabangan Akses:

- **Jika `role === "admin"`:** Panggil `next()`, request diloloskan menuju eksekusi utama (misal: tambah produk/hapus data).
- **Jika `role !== "admin"`:** Server mengembalikan response `403 Forbidden` (Akses Ditolak), sehingga kode di endpoint utama tidak akan pernah dieksekusi

```
JavaScript
app.post('/api/products', autentikasi_user, Authorization, (req, res) => {
  // Handler tambah produk (Hanya berjalan jika lolos 2 middleware di atas)
});
```